Brix Build Layers

Design note — composing incremental, cache, and atomic capabilities over a partially unrealized execution plan · July 2026

1. Context

The Brix spec (`freshStart.md`) composes build capabilities by nesting builders:

```
AtomicBuilder(
    CacheBuilder(
        IncrementalBuilder(change_set, create_graph),
        cache_dir="build/cache",
    ),
    staging_dir="build/staging",
)
```

and describes the wrapper layers as plan transforms: `ExecutionPlan → ExecutionPlan`. The root builder lowers the logical graph into a plan; each wrapper transforms the plan it receives from the builder it wraps.

Brix plans, however, are **partially unrealized**. An `ExpandAll` node is an expansion point, not a loop: it sits in the plan as a single unrealized node until its collection membership is known, then expands into concrete sibling process nodes *during execution*. Generated files make this unavoidable — the members of `build/generated/{name}.cpp` do not exist until the generator action runs. This is already implemented and working in the prototype's scheduler.

2. The problem: static transforms miss runtime nodes

A one-shot plan transform runs after `plan()` and before execution. At that moment the plan for the canonical C++ example contains exactly two nodes: the unrealized `ExpandAll` and `link app`. The concrete compile actions — the nodes caching and staging exist *for* — are created later, inside the executor, after every layer's transform has already run.

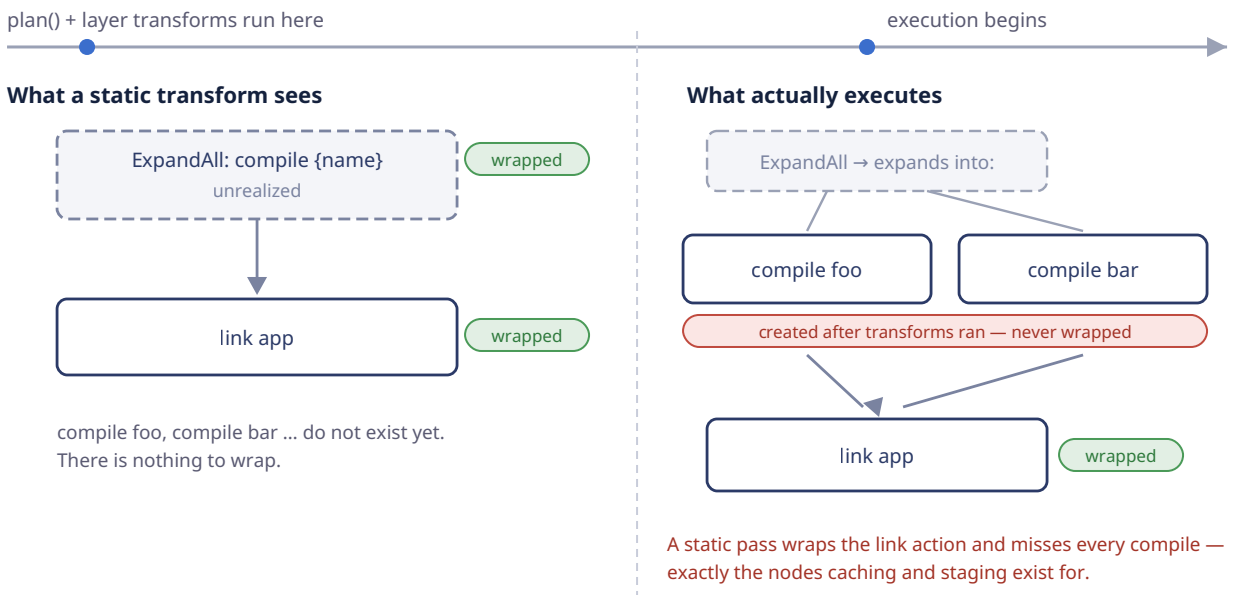


Figure 1 — The static-transform trap: nodes created by runtime expansion bypass any one-shot plan rewrite.

The conclusion: capability decisions cannot all be applied to the plan before execution. They have to bind *during* execution. The question is what shape that takes without each layer re-implementing scheduler concerns.

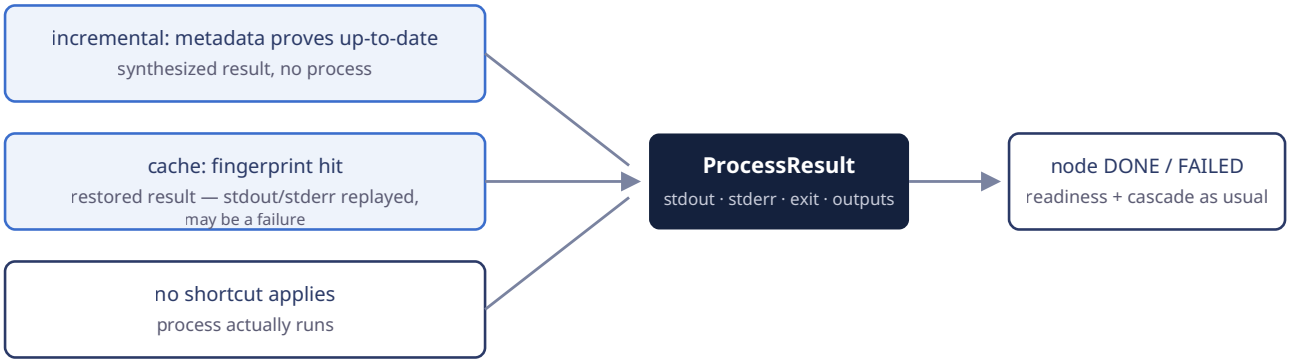
3. The principle: lazy decisions, eager graph

One tempting shape is to make layers *node sources*: the scheduler pulls ready nodes from the outermost layer, which pulls from inner layers and transforms what flows through. That works, but it splits dependency bookkeeping across layers — what replaces a dropped node, how edges attach to a substituted node, when downstream readiness fires. Those are scheduler concerns, and distributing them is where composition bugs breed.

> **Design rule.** Keep the graph eager and centralized; make the *decisions* lazy. The scheduler owns nodes, edges, readiness, and failure cascade. A layer never adds or removes nodes — it decides **how a node reaches its result**, at the moment the node becomes ready.

Node-ready time is exactly when the decision becomes possible: all inputs exist, so fingerprints and cache keys can finally be computed. And every capability's "shortcut" turns out to be the same move — **complete the node with a result obtained somewhere other than a process**:

- **Incremental skip** — metadata proves the node is up to date; synthesize its previous result.
- **Cache hit** — restore the stored result, replaying stdout/stderr; a cached *failure* restores as a failure and flows into the normal cascade-skip path untouched.
- **No shortcut** — the process actually runs.

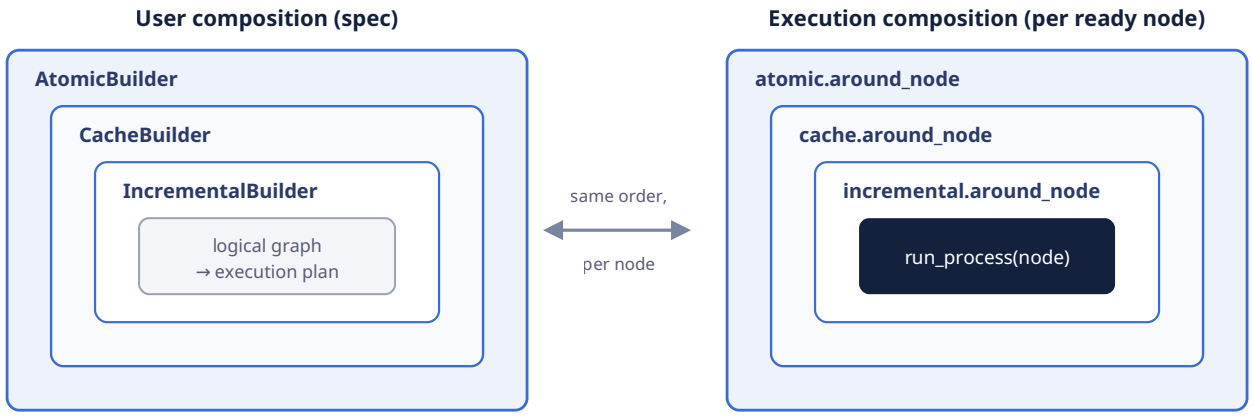


Downstream nodes cannot tell which path produced the result — only acquisition varies.

Figure 2 — Every capability is a different way of acquiring the same thing. Node identity and graph structure never change.

4. The design: middleware + events

The shape that satisfies the rule is a **middleware stack around node execution**, plus a small set of build-scoped events. The executor's current hardwired `_run_process_node` becomes the innermost callable; each layer wraps it. The stack order mirrors the user-facing builder nesting exactly, so the spec's composition story is preserved — it is just applied per node at run time instead of once up front.



Runtime-expanded nodes get the identical stack for free — there is no "transform already happened" moment to miss.

Figure 3 — Builder nesting and the per-node middleware stack are the same composition, applied at different times.

Layer interface sketch

```
class BuildLayer:
    def on_build_begin(self, plan, ctx): ...
        # atomic: prepare staging dir, or recover an interrupted build

    def around_node(self, node, ctx, inner) -> ProcessResult:
        # incremental: fingerprint fresh?  -> synthesize up-to-date result, never call inner
        # cache:       key hit?            -> restore result;  else result = inner(ctx); store
        # atomic:      ctx.output_root = staging;  result = inner(ctx)
        return inner(ctx)

    def on_collection_realized(self, expand_node, members, ctx): ...
        # incremental: diff previous vs current membership -> stale-output deletions

    def on_build_end(self, result, ctx): ...
        # atomic: all ok -> commit staged adds/deletes; else leave staged for status/recovery
```

```
# executor, when a node becomes ready:
call = lambda ctx: run_process_node(node, ctx)
for layer in [incremental, cache, atomic]:     # innermost first — atomic ends up outermost
    call = wrap(layer.around_node, node, call)
result = call(ctx)                              # scheduler records DONE/FAILED as today
```

5. Per-node decision flow

Walking one node through the full `atomic(cache(incremental(run)))` stack:

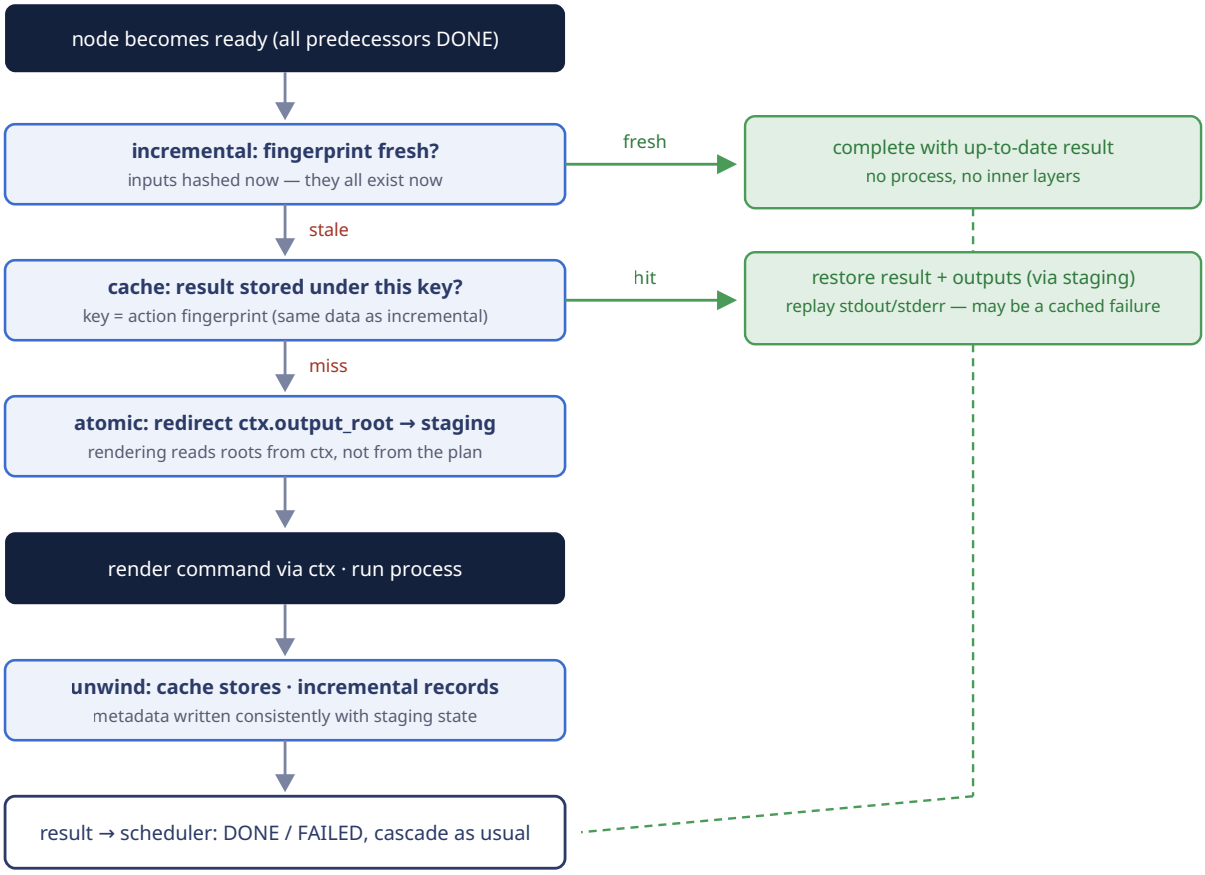


Figure 4 — One node's path through the stack. Green paths are shortcuts; all paths end at the same scheduler hand-off.

Note the cross-layer couplings this makes explicit rather than hiding: the cache key and the incremental fingerprint are essentially **the same computation** (command template, tool identity, input content hashes), so they should share one fingerprint function. And because atomic is outermost, cache-restored outputs land in staging exactly like process-written outputs — the spec's requirement falls out of stack order instead of special-casing.

6. What does not fit in around_node

Two capability behaviors are not per-node decisions, which is why the interface has events beyond the middleware method.

Build-scoped: the atomic commit

Staged additions and deletions commit only if the whole build succeeds (`on_build_end`). If the build fails or crashes, real outputs remain untouched and the staged state stays inspectable for `status` and recovery (`on_build_begin` checks for it). Incremental metadata must be written in agreement with the commit: if the commit never happens, metadata must not claim the outputs exist — otherwise the *next* incremental build is silently wrong.

Membership-scoped: deleted-source cleanup

When a collection realizes, the incremental layer compares current membership against the previous build's persisted membership. A vanished member means its outputs must be deleted and downstream consumers rebuilt — file hashes alone can never detect this.

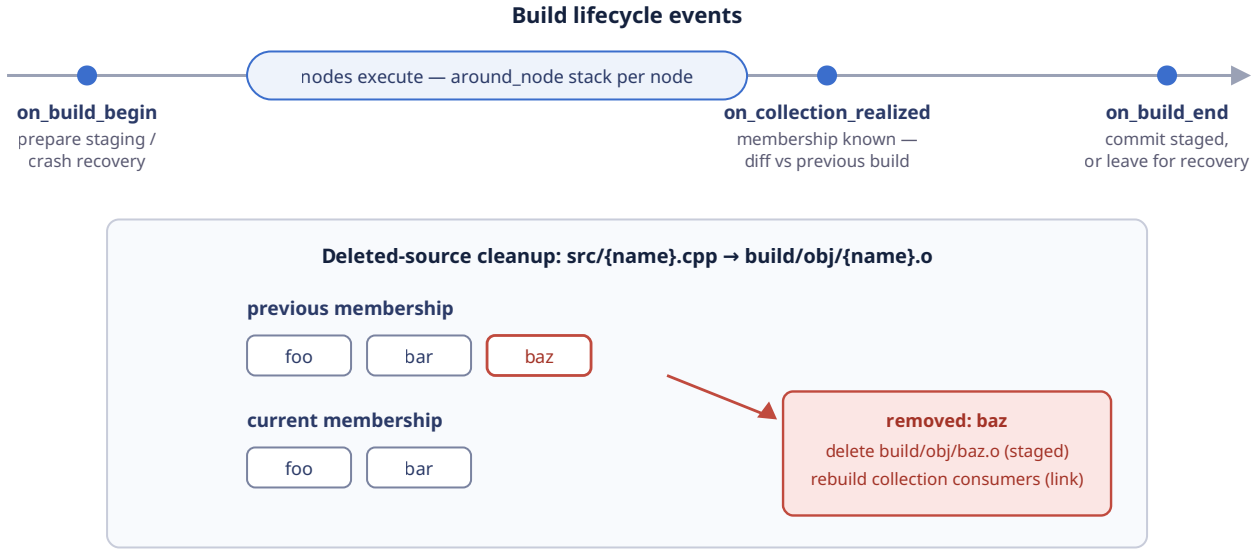


Figure 5 — Lifecycle events, and the membership diff that per-node decisions can never see.

7. The render context

Rendering currently resolves paths directly against the plan's workspace root. The atomic layer needs declared outputs to resolve into `build/staging/add/…` while inputs still resolve into real locations, then needs deletions expressed as staged markers. So the render step takes its roots from a small context object that layers may modify:

```
class NodeContext:
    workspace_root: Path      # where inputs are read and collections realize
    output_root: Path         # where declared outputs are written (atomic: staging)
    # later: env, scratch dirs, per-node metadata handles
```

This is the one place a capability legitimately reaches "inside" node execution, and making it an explicit, layer-visible object keeps that reach narrow and testable. It is also a seam the Rust engine keeps: the context is plain data across the FFI boundary.

8. Validation plan

Step	What	What it proves
1	Introduce `BuildLayer`, `NodeContext`, and the stack in the executor; `CleanBuilder` runs with an empty stack.	The seams exist without changing behavior — full test suite stays green.
2	A trivial layer (logging or dry-run) threaded through the stack.	Composition works, including for runtime-expanded nodes; cheapest possible proof.
3	Persistent metadata + `TimeAndHashChangeDetector`; `IncrementalBuilder` as the first real layer.	Fingerprints, membership persistence, deleted-source and failed-output cleanup — the correctness core.
4	`CacheBuilder`, sharing the fingerprint function.	Result restore and failure replay through the same seam.
5	`AtomicBuilder`: staging via `ctx.output_root`, commit in `on_build_end`.	The most invasive capability lands last, on proven seams.

9. Spec design rules, preserved

- **The logical graph is clean-build truth** — layers never touch user declarations; they intercept execution.
- **Capabilities layer around the graph** — builder nesting is honored; it becomes the middleware order.
- **ExpandAll expands into schedulable nodes** — unchanged, and layers apply to expanded nodes uniformly.
- **Failed results are real results** — a restored failure is just a `ProcessResult` with a nonzero exit; cascade behavior is identical.
- **Cache and atomicity are independent** — neither knows the other exists; stack order alone routes restored outputs through staging.
- **Rust eventually owns the engine** — scheduler, stack dispatch, fingerprints, metadata, and `NodeContext` are all below the DSL boundary; layers are engine policies, not user-language features.